

BeatByte V2 – Full Project Report

Project Overview

BeatByte V2 is a full-stack music streaming web application inspired by modern music platforms such as Spotify. The application allows users to create accounts, log in securely, browse songs, play music, search songs, explore artists, like songs, and maintain a personalized music library.

The project was developed as a portfolio-level full-stack application using the MERN ecosystem and modern frontend technologies. It demonstrates user authentication, state management, REST API integration, responsive UI design, cloud database integration, and deployment practices.

Project Objectives

The primary objectives of BeatByte V2 were:

- Build a modern music streaming interface.
 - Implement secure user authentication.
 - Create a responsive user experience across desktop and mobile devices.
 - Develop a reusable music player component.
 - Manage user-specific data such as liked songs and recently played tracks.
 - Deploy a production-ready full-stack application.
-

Tech Stack

Frontend

- React.js
- React Router DOM
- React Icons
- Context API
- CSS3
- Glassmorphism UI Design
- Vite

Backend

- Node.js
- Express.js

Database

- MongoDB Atlas
- Mongoose ODM

Authentication

- JWT (JSON Web Token)
- bcrypt.js

Deployment

Frontend

- Vercel

Backend

- Render

Database

- MongoDB Atlas
-

System Architecture

BeatByte follows a three-tier architecture:

Presentation Layer

React Frontend

Responsible for:

- UI rendering
- Routing
- State management
- API communication

Application Layer

Express Backend

Responsible for:

- Authentication
- User management
- Song APIs
- Request handling

Data Layer

MongoDB Atlas

Responsible for:

- User storage
 - Song storage
 - Metadata management
-

Features Implemented

1. User Authentication

Registration

Users can:

- Create an account
- Enter name
- Enter email
- Enter password

Data is stored securely in MongoDB.

Login

Users can:

- Log in using credentials
- Receive authentication response
- Access protected pages

Logout

Users can:

- Clear session data
 - Return to login page
-

2. Home Dashboard

The Home page displays:

- Personalized welcome message
- Trending Songs section
- Featured Artists section

The dashboard dynamically fetches songs from MongoDB through backend APIs.

3. Music Player

A custom-built music player was implemented.

Features

Play Song

Users can:

- Select any song
- Start playback instantly

Pause Song

Users can pause currently playing music.

Progress Bar

Displays:

- Current playback position
- Total duration

Users can click anywhere on the progress bar to seek.

Next Song

Automatically plays next track.

Previous Song

Returns to previous track.

Auto Play

When a song ends:

- Next song starts automatically
-

4. Artist Section

Featured artists are displayed on the home page.

Users can:

- Click an artist
- Navigate to artist-specific page

- View songs belonging to that artist

Implemented Artists

Examples:

- Arijit Singh
 - KK
 - Jubin Nautiyal
 - Atif Aslam
-

5. Search Functionality

Users can:

- Search songs
- Filter results instantly
- Browse matching tracks

Search updates dynamically without page refresh.

6. Liked Songs System

Users can:

- Like songs
- Remove liked songs
- View all liked songs in Library

Important Enhancement

Liked songs are stored per user.

Example:

User A: likedSongs_userA@gmail.com

User B: likedSongs_userB@gmail.com

This prevents users from seeing each other's data.

7. Recently Played System

Tracks played by a user are stored automatically.

Features

- Recent listening history
- Automatically updated
- Maximum history maintained

Important Enhancement

Recently played history is user-specific.

Each user has an independent listening history.

8. Library Section

The Library page displays:

Liked Songs

Shows:

- Total liked songs
- Song cards
- Play functionality

Recently Played

Shows:

- Listening history
 - Track count
 - Quick access to recent tracks
-

9. Responsive Design

BeatByte was optimized for:

Desktop

- Sidebar navigation
- Wide content layout

Mobile

Implemented:

- Bottom navigation bar

- Spotify-inspired layout
- Responsive player controls
- Responsive library view

Mobile users can access:

- Home
- Search
- Library
- Profile

without losing functionality.

10. Modern UI Design

The application uses:

Glassmorphism Effects

Implemented:

- Blur effects
- Transparent cards
- Frosted glass appearance

Dark Theme

Features:

- Modern music-app aesthetic
 - Improved readability
 - Better visual appeal
-

Database Design

User Schema

Fields:

- Name
- Email
- Password
- Created At
- Updated At

Song Schema

Fields:

- Title
 - Artist
 - Cover Image
 - Audio URL
 - Created At
 - Updated At
-

API Endpoints

Authentication

POST /api/users/register

Creates new user account.

POST /api/users/login

Authenticates existing user.

Songs

GET /api/songs

Returns all songs.

POST /api/songs

Creates new song entry.

Challenges Faced

1. Music Player State Management

Issue:

- Player UI not updating

Solution:

- Centralized state using Context API

2. Song Synchronization

Issue:

- Selected song not reflected in player

Solution:

- Shared PlayerContext
-

3. User Data Leakage

Issue:

- Different users seeing same liked songs

Solution:

- User-specific localStorage keys
-

4. Deployment Errors

Issue:

- Login/Register failing

Root Causes:

- Wrong API routes
- CORS configuration
- MongoDB Atlas whitelist

Solutions:

- Fixed endpoint URLs
 - Configured CORS
 - Added Atlas network access
-

5. Mobile Navigation

Issue:

- Sidebar hidden on mobile

Solution:

- Built bottom navigation system
-

Security Measures

Implemented:

- Password hashing using bcrypt
 - JWT authentication
 - Protected routes
 - Input validation
 - Secure database storage
-

Deployment Architecture

Frontend:

Vercel

Backend:

Render

Database:

MongoDB Atlas

The application is fully cloud-hosted and accessible online.

Learning Outcomes

Through BeatByte V2, the following skills were developed:

- Full Stack Development
- React State Management
- REST API Development
- Authentication Systems
- MongoDB Integration
- Deployment Workflows
- Responsive Design
- Context API
- UI/UX Design Principles
- Cloud Hosting

Future Improvements

Potential enhancements include:

- Playlist Creation
- Playlist Sharing
- Admin Dashboard
- Upload Songs Feature
- User Profile Editing
- Audio Visualizer
- Lyrics Integration
- Queue Management
- Real-Time Streaming
- Recommendation System
- AI-Powered Music Suggestions

Conclusion

BeatByte V2 is a modern full-stack music streaming application developed using React, Node.js, Express, MongoDB Atlas, and modern deployment platforms. The project successfully demonstrates authentication, music playback, user personalization, responsive design, API integration, cloud deployment, and scalable application architecture.

The application serves as a strong portfolio project showcasing practical full-stack development skills and industry-relevant technologies.